

# Multi-Agent Drone Formations

A 23 drone squad flies a coordinated formation (wedge/line/column/split) from a single natural-language operator command, simulated in [gym-pybullet-drones](#).

```
operator prompt -> local LLM (Ollama) -> validated JSON mission plan
                -> formation layer -> deterministic executor -> PyBullet sim
```

## Overview

The LLM's only job is turning operator intent into a schema-valid JSON mission plan. It never touches a drone directly. See [docs/architecture.md](#) for the full diagram and boundary contract, [docs/design\\_decisions.md](#) for the reasoning behind every non-obvious choice in this codebase (two-stage validation, the formation offset math, why structured outputs alone isn't the safety mechanism, and more), and [docs/WRITEUP.md](#) for the submission write-up (approach, challenges attempted, scaling story, cited sources).

**LLM backend:** this runs on a locally-hosted model via [Ollama](#) (default `qwen2.5:7b`), not the Anthropic API see “Ollama Setup” below and [docs/design\\_decisions.md](#) §14 for why, and what switching back would take. Nothing outside `llm_planner/planner.py` knows or cares which LLM produced the mission JSON.

**Verified status:** this codebase was originally written in an environment with no local Python interpreter or Docker available (every `gym-pybullet-drones` API detail was checked against upstream source rather than recalled from memory - see [docs/design\\_decisions.md](#)). It has since been run end to end on Linux: `pytest` passes, `docker build + docker run` (headless, `--mission-file`) works, and the full `--prompt` → Ollama → validate → formation → `--gui` PyBullet path has been exercised repeatedly and recorded - see [demo/](#) for the videos.

## Project Layout

|              |                                                                                                              |
|--------------|--------------------------------------------------------------------------------------------------------------|
| llm_planner/ | Calls a local Ollama model over HTTP, returns raw JSON.<br>Only file that talks to an LLM.                   |
| schema/      | mission_schema.json (the contract) + validator.py (the safety gate)                                          |
| formation/   | wedge/line/column geometry, route-splitting + regroup logic                                                  |
| executor/    | validated JSON + formation output -> sim inputs<br>(deterministic, no LLM)                                   |
| sim/         | gym-pybullet-drones integration<br>(CtrlAviary + DSLPIDControl + Logger)                                     |
| docs/        | architecture diagram, design-decision rationale,<br>submission write-up                                      |
| demo/        | example prompts, pre-generated example mission JSON<br>(no LLM needed), and the two recorded gui demo videos |
| tests/       | unit tests for the schema validator and<br>formation-splitting logic                                         |
| run.py       | CLI entrypoint                                                                                               |
| Dockerfile   | builds and runs the whole thing on a clean machine                                                           |

## Ollama Setup

*Required for --prompt; skip if you only use --mission-file.*

Ollama runs **on the host**, not inside Docker - it is a separate service that both the local Python path and the Docker container talk to over HTTP.

```
# Install (Linux; sets up a systemd service running
# `ollama serve` on 127.0.0.1:11434)
curl -fsSL https://ollama.com/install.sh | sh

# Confirm the server is up
curl http://localhost:11434/api/version # -> {"version": "..."}

# Pull the model (~4.7GB; already instruction-tuned,
# no separate -instruct tag)
ollama pull qwen2.5:7b
```

### Why qwen2.5:7b

It is specifically strong at structured/JSON output relative to its size, and small enough to run comfortably on a normal desktop (~5GB disk, ~8GB RAM) rather than needing the 40GB+ of RAM/VRAM a 70B-class model would want - which is overkill anyway for a narrow, schema-constrained “fill in this JSON” task, not open-ended reasoning. llama3.1:8b is a fine alternative (OLLAMA\_MODEL=llama3.1:8b); it just tends to need a bit more corrective prompting to stay JSON-only, which is exactly what the retry loop in llm\_planner/planner.py is for.

*Not on Linux?* macOS: `brew install ollama` or the app from ollama.com. Windows: the installer from ollama.com, or use WSL2 and follow the Linux steps above (WSL2 is also what makes the Docker networking story below match the Linux case, not the macOS/Windows Docker Desktop case).

## Install

Docker (recommended - this is what “clean machine” testing means here)

```
docker build -t drone-formations .

# Offline demo, no Ollama needed (the default CMD):
docker run rm -v "$(pwd)/logs:/app/logs" drone-formations

# With the LLM (Linux see "Docker + Ollama networking" below
# for why network host is the flag to use here, not
# host.docker.internal):
docker run rm network host \
  -v "$(pwd)/logs:/app/logs" \
  drone-formations prompt "You three sweep this area in a wedge formation"
```

-v "\$(pwd)/logs:/app/logs" bind-mounts the telemetry output directory so CSV logs land on your host, not just inside the (disposable) container.

## Docker + Ollama Networking

Ollama listens on the **host's** 127.0.0.1:11434 by default. The container needs to reach that, and the reliable way to do it differs by platform.

| Platform           | Flag                        |
|--------------------|-----------------------------|
| Linux (incl. WSL2) | <code>--network host</code> |

`llm_planner/planner.py` reads `OLLAMA_BASE_URL` (default `http://localhost:11434`), so on Linux with `--network host` no override is needed - the default already resolves correctly, since “localhost” inside the container is genuinely the host’s localhost in that mode.

## Local (non-Docker) Development

Requires Python 3.10+ (matches `gym-pybullet-drones`’ own requirement).

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
# No API key needed anywhere just `ollama serve` running
# (see Ollama Setup above).
```

`requirements.txt` installs `gym-pybullet-drones` directly from GitHub (pinned to a specific commit - it isn’t published on PyPI; see `docs/design_decisions.md` §13 for why that commit and not a release tag).

## Reproducibility Notes

*Read this before running `--gui`.*

There are two genuinely different ways to run this, and they have been verified to different degrees.

- **Docker, headless, `--mission-file`** (the default CMD): fully self-contained, no Ollama, no GUI, no host dependencies beyond Docker itself. This is the path verified to “just work” on a clean Linux machine - `docker build` then `docker run` with no extra setup.
- **Local + Ollama + `--gui`**: this is the path the two demo videos in `demo/` actually show, and it has two things a Docker-only reviewer should know before trying it:
  1. **Ollama is not in the Docker image and must be installed separately on the host** (see “Ollama Setup” above) - it is a real service (`ollama serve`) that both the local Python path and, if you use `--network host`, the container itself talk to over HTTP. There is no way to get the `--prompt` path working from Docker alone.
  2. **CPU-only local inference is slow, on purpose left un-hidden rather than silently eaten by a long-feeling hang.** On the machine used to record the demo videos (CPU-only, no GPU), a single schema-constrained `qwen2.5:7b` generation (Stage 1) took roughly **2 minutes**. That is why `llm_planner/planner.py` prints “Calling local LLM, this may take 1-3 minutes on CPU...” before the call, and why `OLLAMA_TIMEOUT_SECONDS` defaults to 120s rather than a typical API timeout of a few seconds. If you run `--prompt ... --gui` yourself and it sits for a while before Stage 1 prints its output, that is expected, not a hang. A GPU host or a hosted API (see `docs/WRITEUP.md`’s scaling section) would cut this to low-single-digit seconds without any code change beyond `llm_planner/planner.py`.

`--mission-file` sidesteps both of these entirely (no Ollama call at all), which is why it is the Docker CMD’s default and the fastest way to see Stages 24 work.

## Usage

```
# Full pipeline: prompt -> Ollama (local LLM) -> validated JSON
# -> formation -> sim
# (requires `ollama serve` running and the model pulled see
# Ollama Setup above)
python run.py prompt "You three sweep this area in a wedge formation"

# Skip the LLM call entirely validate + execute a pre-generated
# mission (useful for offline testing, demoing the executor without
# Ollama running, or iterating on the executor/sim without waiting
# on model inference)
python run.py mission-file demo/example_missions/wedge_sweep.json

# Visual demo with the PyBullet GUI (local only not for
# Docker/headless)
python run.py mission-file demo/example_missions/split_patrol.json gui

# Override the model (defaults to qwen2.5:7b; OLLAMA_MODEL env
# var also works)
python run.py prompt "..." model llama3.1:8b
```

More example prompts: `demo/example_prompts.txt`.

## Exact Commands From the Demo Videos

These are the two commands actually run to produce the recordings in `demo/` (see “Demo Videos” below) - both use `--prompt` (a live Ollama call, not `--mission-file`) with `--gui`, and both require `ollama serve` running with `qwen2.5:7b` pulled.

```
# Wedge-formation sweep
python3 run.py prompt "You three sweep this area in a wedge formation" gui

# Split-route-and-regroup
python3 run.py prompt "Split this patrol route between the two of you and regroup at
↪ the end" gui
```

## Example Prompts and What They Produce

| Prompt                                                                  | formation.type | task.type                   |
|-------------------------------------------------------------------------|----------------|-----------------------------|
| “You three sweep this area in a wedge formation”                        | wedge          | area_sweep                  |
| “Split this patrol route between the two of you and regroup at the end” | split          | patrol_route                |
| “Fly a tight line formation along the perimeter”                        | line           | patrol_route                |
| “Two drones, column formation, loop the access road three times”        | column         | patrol_route, loop_count: 3 |

## Expected Output

`run.py` prints every stage so the pipeline is auditable end to end.

```
=====
STAGE 1: LLM PLANNING
=====
Operator prompt: 'You three sweep this area in a wedge formation'
Raw LLM output:
{ ... mission JSON ... }

=====
STAGE 2: VALIDATION (schema + safety)
=====
VALID. Mission accepted:
{ ... }

=====
STAGE 3: FORMATION + EXECUTION (deterministic, no LLM involved)
=====
Per-drone waypoints (formation layer output, meters, local XY frame):
  drone_0: (0.0, 30.0), (120.0, 30.0)
  drone_1: (-8.0, 38.0), (112.0, 38.0)
  drone_2: (-8.0, 22.0), (112.0, 22.0)

=====
STAGE 4: SIMULATION RESULT
=====
Agents simulated: ['drone_0', 'drone_1', 'drone_2']
Control steps run: 1440
Minimum observed inter-drone separation: 10.87 m (required: 3 m)
Telemetry written to: logs/
```

On rejection (bad schema, unsafe parameters, or a post-formation separation violation), the pipeline stops and prints exactly what failed instead of executing anything.

```
=====
STAGE 2: VALIDATION (schema + safety)
=====
REJECTED. Mission will not be executed. Errors:
  - altitude_m: altitude_m (100) exceeds safety.max_altitude_m (50).
```

## Demo Videos

`demo/` contains two screen recordings of the full pipeline running with `--gui`, LLM planning done live by Ollama (not `--mission-file`).

| Video                                                  | Prompt                  | Command                                                                                                          |
|--------------------------------------------------------|-------------------------|------------------------------------------------------------------------------------------------------------------|
| <a href="#">Screencast 10:45:35 AM.webm</a> (~2.5 min) | Wedge-formation sweep   | <pre>python3 run.py --prompt "You three sweep this area in a wedge formation" --gui</pre>                        |
| <a href="#">Screencast 11:55:39 AM.webm</a> (~4 min)   | Split-route-and-regroup | <pre>python3 run.py --prompt "Split this patrol route between the two of you and regroup at the end" --gui</pre> |

Both show the operator prompt going in, Stage 14 console output printing live, and the PyBullet GUI executing the resulting formation.

## Testing

```
pip install -r requirements.txt # or just: pip install jsonschema pytest
pytest tests/
```

The test suite covers `schema/validator.py` and `formation/` - the deterministic, auditable parts of the system - entirely without touching the LLM or the simulator, so it runs in well under a second and needs no API key or PyBullet (49 tests, confirmed passing).

**If pytest throws an `INTERNALERROR` instead of running tests:** this is not a bug in this project - it means some *other* globally-installed pytest plugin on your machine is incompatible with the installed pytest version and is being auto-loaded (e.g. a ROS 2 `launch_testing` plugin registered in `~/.local/lib/.../site-packages` was observed causing exactly this on the development machine). Using the venv from “Install” above avoids picking up global site-packages plugins; if you’re intentionally running outside a venv, `PYTEST_DISABLE_PLUGIN_AUTOLOAD=1` `pytest tests/` sidesteps it.

## Configuration

| Env var                             | Purpose                                                                                                                                                     |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>OLLAMA_MODEL</code>           | Overrides the default model ( <code>qwen2.5:7b</code> ). Same as <code>--model</code> . Not needed for <code>--mission-file</code> .                        |
| <code>OLLAMA_BASE_URL</code>        | Overrides the default Ollama server URL ( <code>http://localhost:11434</code> ). Needed on macOS/Windows Docker Desktop - see “Docker + Ollama Networking.” |
| <code>OLLAMA_TIMEOUT_SECONDS</code> | Overrides the default per-request timeout (120s) for local model inference.                                                                                 |

No API key of any kind is required anywhere in this project.

## Attribution

- Simulation built on [gym-pybullet-drones](#) (MIT License, Copyright © 2020 Jacopo Panerati). `sim/sim_runner.py` adapts the `CtrlAviary/DSLPIDControl/Logger` control-loop pattern from the project's own `examples/pid.py` - see that file's docstring and `docs/design_decisions.md` §13 for exactly what is cited vs. original.
- LLM planning via a local model served by [Ollama](#) (default `qwen2.5:7b`) - substituted for the Anthropic API due to no available API credit; see `docs/design_decisions.md` §14.

## Known Limitations / Extension Points

Documented in detail in `docs/design_decisions.md`; summarized here.

- `area_sweep` produces a single-pass sweep line, not full multi-lane lawnmower coverage clipped to the polygon (§7).
- Separation/geofence checks operate on waypoints, not continuous-time trajectories between them (§4).
- No heading/yaw concept in the schema - every drone flies level, facing  $+x$ .
- This challenge scopes `num_agents` to 23, matching the schema's `minimum/maximum`.